

Expense Tracker API

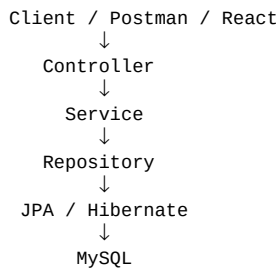
Backend Notes v1 • Easy Hinglish • Beginner → Interview Ready

Important: Yeh notes hamare Expense Tracker project ke concepts aur final workflow ko explain karte hain. Code snippets ko learning ke liye use karo, aur apne current IntelliJ project ko source of truth rakho.

Goal: sirf code yaad karna nahi, balki samajhna ki request Controller se Service, Repository aur MySQL tak kaise jaati hai.

1. Project kya hai?

Expense Tracker API ek Spring Boot backend hai jisme user income aur expenses ko manage kar sakta hai. Humne CRUD, analytics, validation, custom exception handling, filtering, pagination, Swagger/OpenAPI aur CORS implement kiya hai. Is project ka future frontend React + Vite hoga. Frontend HTTP requests ke through isi backend ko call karega.



2. Technology Stack

- Java: application ki programming language.
- Spring Boot: backend application ko quickly build aur run karne ka framework.
- Spring Web: REST APIs banane ke liye.
- Spring Data JPA: database operations ko Java repository ke through handle karne ke liye.
- Hibernate: JPA ka implementation, jo Java objects aur database ke beech mapping karta hai.
- MySQL: actual relational database.
- Maven: dependencies aur build management.
- IntelliJ IDEA: development.
- DBeaver: MySQL/database ko inspect aur SQL run karne ke liye.
- Postman: APIs ko manually test karne ke liye.
- Swagger/OpenAPI: interactive API documentation ke liye.

3. Package Structure aur har layer ka role

Project ko responsibilities ke hisaab se divide kiya gaya hai. Isse code readable aur maintainable rehta hai. Controller → HTTP request/response Service → application/business logic Repository → database access Transaction → database entity DTO → selected/custom response shape Exception → errors ko consistently handle karna Config → application configuration, jaise CORS

```
com.yash.expensetracker
├── Controller
│   ├── TransactionController
├── Repository
│   ├── TransactionRepository
├── Service
│   ├── TransactionService
├── Transaction
│   ├── Transaction
├── dto
│   ├── CategoryExpense
│   ├── MonthlyExpense
├── exception
│   ├── GlobalExceptionHandler
│   ├── TransactionNotFoundException
├── config
│   ├── CorsConfig
└── ExpensetrackerApplication
```

4. Spring Boot main class

@SpringBootApplication main Spring Boot class par lagta hai. Application start karte waqt Spring Boot component scanning aur auto-configuration ke through application ke components ko discover karta hai.

```
@SpringBootApplication
public class ExpensetrackerApplication {
    public static void main(String[] args) {
        SpringApplication.run(
            ExpensetrackerApplication.class, args
        );
    }
}
```

5. Entity: Transaction

Transaction entity hamare data ka main model hai. @Entity Spring Data JPA/Hibernate ko batata hai ki class database table se map hogi. @Id primary key identify karta hai. @GeneratedValue IDs ko automatically generate karne deta hai. Money ke liye BigDecimal use karna better hai kyunki floating-point types money calculations mein precision issues de sakte hain. Transaction date ke liye LocalDate use hua hai.

```
@Entity
public class Transaction {
    @Id
    @GeneratedValue
    private Long id;

    private BigDecimal amount;
    private String category;
    private String type;
    private LocalDate transactionDate;
    private String description;
}
```

6. Validation

Validation ka purpose hai invalid request ko database tak pahunchne se pehle reject karna. @Positive → amount zero se greater hona chahiye. @NotBlank → string null/empty/whitespace-only nahi honi chahiye. @NotNull → value null nahi ho sakti. @Pattern → type sirf EXPENSE ya INCOME hona chahiye. Important: @Valid Controller mein lagana zaroori hai. Humne POST aur PUT request body par @Valid use kiya.

```
@Positive(message = "Amount must be Greater than zero")
private BigDecimal amount;

@NotBlank(message = "Category cannot be Blank")
private String category;

@NotBlank(message = "Type cannot be Blank")
@Pattern(
    regexp = "EXPENSE|INCOME",
    message = "Type must be EXPENSE or INCOME"
)
private String type;

@NotNull(message = "TransactionDate cannot be null")
private LocalDate transactionDate;
```

7. JSON null vs string "null"

Ek useful debugging lesson: JSON mein `<b>null</b>` aur `<b>"null"</b>` same nahi hain. `<b>null</b>` actual null value hai, jabki `<b>"null"</b>` ek string hai. Isliye @NotNull validation ka expected behavior tab milega jab request mein actual null bheja jaye.

```
"transactionDate": null

// This is a string, not null:
"transactionDate": "null"
```

8. Repository: JpaRepository

TransactionRepository ne JpaRepository<Transaction, Long> extend kiya. Iska matlab Spring Data humein common CRUD methods already provide karta hai, jaise save(), findAll(), findById(), deleteById(), existsById().

```
public interface TransactionRepository
    extends JpaRepository<Transaction, Long> {
}
```

Pagination ke liye bhi JpaRepository already findAll(Pageable) provide karta hai. Isliye duplicate method banana zaroori nahi hai.

9. JPQL aur Analytics Queries

JPQL Java entity aur uske fields par query karta hai. Humne SUM se total income/expenses nikale aur GROUP BY se category/month-wise aggregation ki.

```
@Query("SELECT SUM(t.amount) FROM Transaction t " +
        "WHERE t.type = 'EXPENSE'")
BigDecimal getTotalExpenses();

@Query("SELECT SUM(t.amount) FROM Transaction t " +
        "WHERE t.type = 'INCOME'")
BigDecimal getTotalIncome();
```

Category-wise concept: expense transactions ko category ke basis par group karke har category ka SUM nikalna.

```
SELECT new com.yash.expensetracker.dto.CategoryExpense(
    t.category,
    SUM(t.amount)
)
FROM Transaction t
WHERE t.type = 'EXPENSE'
GROUP BY t.category
```

Monthly concept: YEAR(transactionDate) aur MONTH(transactionDate) se year/month nikala, phir SUM(amount) aur GROUP BY kiya.

10. DTO kya hai aur kyun use kiya?

DTO ka full form Data Transfer Object hai. DTO tab useful hai jab humein entity ka pura object return nahi karna, balki response mein sirf required calculated/selected data dena ho.

CategoryExpense mein category aur totalAmount hota hai. Repository JPQL constructor expression se directly DTO objects create karta hai.

```
public class CategoryExpense {
    private String category;
    private BigDecimal totalAmount;

    public CategoryExpense(
        String category,
        BigDecimal totalAmount) {
        this.category = category;
        this.totalAmount = totalAmount;
    }
}
```

MonthlyExpense mein year, month aur totalAmount hota hai. Yeh frontend charts ke liye useful response shape hai.

11. Service Layer

Service layer Controller aur Repository ke beech bridge hai. Controller ko database details se isolate karna aur business/application logic ko ek jagah rakhna iska major benefit hai.

```
public BigDecimal getBalance() {
    BigDecimal income = getTotalIncome();
    BigDecimal expenses = getTotalExpenses();
    return income.subtract(expenses);
}
```

Yahan balance = total income – total expenses. Agar income expenses se kam ho, balance negative ho sakta hai.

12. CRUD APIs

```
POST    /transactions
GET     /transactions
GET     /transactions/{id}
PUT     /transactions/{id}
DELETE  /transactions/{id}
```

POST create karta hai. **GET** read karta hai. **PUT** existing transaction update karta hai.

DELETE remove karta hai.

Request body JSON ko `@RequestBody` se Java object mein convert kiya jata hai. URL ke andar ID ko `@PathVariable` se read kiya jata hai.

13. Filtering

Filtering endpoint optional type aur category accept karta hai:

```
GET /transactions/filter?type=EXPENSE
GET /transactions/filter?category=Food
GET /transactions/filter?type=EXPENSE&category=Food
GET /transactions/filter

@Query("""
    SELECT t FROM Transaction t
    WHERE (:type IS NULL OR t.type = :type)
    AND (:category IS NULL OR t.category = :category)
    """)
List<Transaction> findByFilters(
    @Param("type") String type,
    @Param("category") String category
);
```

Logic: agar parameter null hai, us filter ko ignore karo. Agar value present hai, matching records return karo. Is pattern ki wajah se ek endpoint multiple filter combinations support karta hai.

14. Pagination

Agar database mein thousands of transactions ho jaayen, to ek saath sab records bhejna inefficient ho sakta hai.

Pagination response ko smaller pages mein divide karta hai.

```
GET /transactions/page?page=0&size=10
GET /transactions/page?page=1&size=5
```

Page numbering **0** se start hoti hai. Isliye page=0 first page hai, page=1 second page hai.

```
public Page<Transaction> getTransactionsWithPagination(
    int page, int size) {

    Pageable pageable = PageRequest.of(page, size);

    return transactionRepository.findAll(pageable);
}
```

Spring Page response mein content ke saath `pageNumber`, `pageSize`, `totalElements`, `totalPages`, `first`, `last` jaise metadata aa sakte hain. Frontend isi metadata se Next/Previous UI bana sakta hai.

15. Exception Handling

Normal CRUD code mein missing ID ke liye null return karne ke bajay humne custom `TransactionNotFoundException` banaya. Isse missing resource ko clearly 404 response mein convert kiya ja sakta hai.

```
public class TransactionNotFoundException
    extends RuntimeException {

    public TransactionNotFoundException(String message) {
        super(message);
    }
}
```

`GlobalExceptionHandler` par `@RestControllerAdvice` use hua. Isse application ke controllers se aane wali selected exceptions centrally handle ki ja sakti hain.

```
@ExceptionHandler(TransactionNotFoundException.class)
public ResponseEntity<Map<String, String>>
handleTransactionNotFound(
    TransactionNotFoundException ex) {

    Map<String, String> error = new HashMap<>();
    error.put("message", ex.getMessage());

    return ResponseEntity
        .status(HttpStatus.NOT_FOUND)
```

```
        .body(error);  
    }  
}
```

Validation errors ke liye `MethodArgumentNotValidException` handle kiya gaya aur field-wise messages ka map return hota hai. Iska expected status 400 Bad Request hai.

16. HTTP Status Codes jo project mein important hain

Status	Meaning	Project example
200 OK	Request successful	GET /transactions
400 Bad Request	Input invalid	Validation failure
404 Not Found	Resource nahi mila	GET /transactions/9999

17. Swagger / OpenAPI

Swagger UI browser mein interactive API documentation deta hai. Humne controller ko `@Tag` se group kiya aur endpoints ko `@Operation` se describe kiya. Parameters ke liye `@Parameter` use hua.

```
@Tag(  
    name = "Transactions",  
    description = "APIs for managing income and expense transactions"  
)  
  
@Operation(  
    summary = "Get total expenses",  
    description =  
        "Returns the total amount of all expense transactions"  
)
```

Swagger ka benefit: developer ko endpoint, purpose, parameters aur request/response behavior samajhne ke liye alag document search nahi karna padta.

Swagger UI humne local application par `/swagger-ui.html` se open karke verify kiya.

18. CORS

CORS ka full form Cross-Origin Resource Sharing hai. Future React frontend generally `localhost:5173` par run ho sakta hai, jabki Spring Boot backend `localhost:8080` par. Browser cross-origin request rules apply karta hai, isliye backend ko frontend origin allow karna hota hai.

```
@Configuration  
public class CorsConfig {  
  
    @Bean  
    public WebMvcConfigurer corsConfigurer() {  
        return new WebMvcConfigurer() {  
            @Override  
            public void addCorsMappings(CorsRegistry registry) {  
                registry.addMapping("/**")  
                    .allowedOrigins("http://localhost:5173")  
                    .allowedMethods(  
                        "GET", "POST", "PUT", "DELETE"  
                    );  
            }  
        };  
    }  
}
```

`<b>/**</b>` ka meaning: application ke Spring MVC URL mappings par ye CORS rule apply karo. Ye frontend ke future browser requests ko allow karne ke context mein use hua hai.

19. MySQL aur DBeaver

MySQL actual persistent data store hai. DBeaver ek database client hai jisse database, tables aur records inspect kiye ja sakte hain aur SQL commands execute ki ja sakti hain.

Auto-increment IDs ke case mein manually deleted ID normally next insert mein automatically reuse nahi hoti. Isliye normal CRUD flow mein ID ko manually assign karne ke bajay database/JPA ko generate karne dena better hai.

DBeaver mein multiple SQL statements ek saath execute karne par client/settings ke according multi-statement issue aa sakta hai. Humne database commands ko separately execute karke solve kiya.

20. Postman Testing

Postman se humne API requests manually verify ki. Important testing categories:

```
Normal:
GET /transactions

Validation:
POST /transactions
with invalid amount/category/type/date

Not found:
GET /transactions/9999

Filtering:
GET /transactions/filter?type=EXPENSE
GET /transactions/filter?category=Food
GET /transactions/filter?type=EXPENSE&category=Food

Pagination:
GET /transactions/page?page=1&size=5
```

Testing ka main idea hai sirf happy path nahi, balki invalid input aur missing resources bhi check karna.

21. Database Credentials aur Environment Variables ■

Real database password ko source code ya GitHub repository mein commit nahi karna chahiye. Humne MySQL password change karke application.properties mein direct password rakhne ke bajay environment variable reference use kiya.

```
spring.datasource.password=${DB_PASSWORD}
```

IntelliJ Run Configuration mein DB_PASSWORD environment variable set kiya gaya. Isliye GitHub par actual password ki jagah sirf variable reference rehta hai.

Important security lesson: Agar koi credential pehle GitHub par commit ho chuka hai, sirf file se delete karna enough nahi hota. Credential ko rotate/change karna safest step hai. Humne old MySQL password change kiya.

22. Important Errors aur unse kya seekha

Error 1: DBeaver multi-statement issue
Database creation aur SHOW DATABASES ko separately run karna safer tha.

Error 2: "null" vs null
String "null" actual null nahi hai. Actual JSON null bhejne par @NotNull validation expected way mein trigger hui.

Error 3: Existing invalid DB record
Validation annotations future requests ko validate karti hain. Purane database records automatically fix nahi hote. Invalid record ko valid PUT se repair kiya gaya.

Error 4: Missing transaction
Optional.orElse(null) se null response mil sakta tha. Custom TransactionNotFoundException se clear 404 response banaya gaya.

Error 5: Hibernate dialect/JDBC metadata error
Environment-variable setup ke waqt database connection temporarily fail hua. Root issue database connectivity/configuration side par tha, Hibernate message secondary symptom tha. Environment variable configuration correct hone ke baad application normally start hua.

Error 6: Pagination typo
Method name mein spelling mistake thi. Behavior break nahi hua kyunki Controller aur Service dono same typo use kar rahe the, but final cleanup mein correct kiya gaya.

23. Common annotations: quick revision

Annotation	Simple meaning
@SpringBootApplication	Spring Boot application ka main configuration/entry point
@Entity	Class ko JPA entity/database mapping ke liye mark karta hai
@Id	Primary key
@GeneratedValue	ID generation strategy
@RestController	REST HTTP controller
@Service	Service-layer Spring bean
@Repository	Repository/database access layer
@GetMapping	GET endpoint
@PostMapping	POST endpoint
@PutMapping	PUT endpoint
@DeleteMapping	DELETE endpoint
@RequestBody	JSON request body ko object mein bind karta hai
@PathVariable	URL path value read karta hai
@RequestParam	Query parameter read karta hai
@Valid	Bean validation trigger karta hai
@RestControllerAdvice	Global REST exception handling
@ExceptionHandler	Specific exception ka handler
@Query	Custom JPQL query
@Parameter	Swagger parameter documentation

24. Complete API Cheat Sheet

```
CRUD
POST    /transactions
GET     /transactions
GET     /transactions/{id}
PUT     /transactions/{id}
DELETE  /transactions/{id}

FILTERING
GET /transactions/filter
GET /transactions/filter?type=EXPENSE
GET /transactions/filter?category=Food
GET /transactions/filter?type=EXPENSE&category=Food

PAGINATION
GET /transactions/page?page=0&size=10

ANALYTICS
GET /analytics/expenses
GET /analytics/income
GET /analytics/balance
GET /analytics/category
GET /analytics/monthly
```

25. Interview mein project kaise explain karein?

Short answer: "I built a Spring Boot based Expense Tracker REST API using Spring Data JPA and MySQL. It supports CRUD operations for transactions, validation, centralized exception handling, analytics such as total income, expenses, balance, category-wise and monthly expenses, plus filtering and pagination. I also documented the APIs using Swagger/OpenAPI and configured CORS for a future React frontend."

Follow-up: Controller, Service, Repository kyun?
Responsibilities separate karne ke liye. Controller HTTP layer handle karta hai, Service application logic, Repository database access.

Follow-up: BigDecimal kyun?
Money calculations mein better precision/control ke liye.

Follow-up: DTO kyun?
Calculated/selected response data ko entity se separate shape mein return karne ke liye.

Follow-up: @Valid kya karta hai?
Request body ko entity validation rules ke against check karta hai.

Follow-up: 400 vs 404?
400 ka matlab request/input invalid. 404 ka matlab requested resource exist nahi karta.

Follow-up: Pagination?
Large result set ko smaller pages mein return karna, jisse response aur frontend handling efficient hoti hai.

Follow-up: CORS?
Browser ko different origin se API request karne ki permission dene ka controlled mechanism.

Follow-up: JPQL?
JPA entities/fields ke context mein query language, jise Spring Data JPA @Query ke through use kar sakta hai.

26. Beginner Mental Model: ek POST request ka safar

```
POST /transactions
↓
TransactionController
↓
@Valid request body check
↓
TransactionService
↓
TransactionRepository.save(...)
↓
Hibernate / JPA
↓
MySQL
↓
Saved Transaction
↓
JSON response
```

Is flow ko samajh lena project ka sabse important foundation hai. Isi pattern ko GET, PUT, DELETE aur analytics mein modify karke use kiya gaya.

27. Final Backend Checklist

Area	Status
Spring Boot project setup	Done
MySQL + JPA connection	Done
Transaction entity	Done
CRUD APIs	Done
Validation	Done
Global exception handling	Done
Analytics	Done
Filtering	Done

Area	Status
Pagination	Done
Swagger/OpenAPI	Done
CORS	Done
Code cleanup	Done
Database password moved to environment variable	Done
Postman sanity testing	Done
Git commit/push	Done

28. What comes next?

Backend ab stable hai. Next phase mein React + Vite frontend banega. Frontend isi REST API ko call karega. Dashboard mein balance, income, expenses, charts, transaction list, add/edit/delete, filtering aur pagination connect kiye jaayenge.

Final project complete hone ke baad ek separate final PDF banaya ja sakta hai jisme backend + frontend + integration + screenshots + architecture + deployment + GitHub/README + interview preparation sab consolidated form mein hoga.